Software Engineering Lab

Lab 27

Database Constraints

Memindahkan Validasi ke Level Database untuk Menjamin Integritas Data

Lukman / lukman-ss

Validasi Aplikasi vs Database Constraint

Pola check-then-act rentan race condition di bawah konkurensi

MOTIVASI

Validasi aplikasi saja rentan: SELECT → cek → INSERT dapat diinterleave
Test konkuren 20 goroutine: UnsafeStore menghasilkan >1 record dengan email sama
Perbaikan duplikat data di produksi jauh lebih mahal daripada pencegahan

KONSEP

Race condition terjadi saat dua transaksi membaca keadaan sama
Database constraint memindahkan pengecekan ke titik atomik: saat INSERT
UNIQUE constraint menggunakan B-tree index dengan ROW EXCLUSIVE lock

Goroutine A: scan → tidak ketemu → INSERT sukses

Goroutine B: scan → tidak ketemu → INSERT sukses (DUPLIKAT)

SafeStore delegation ke engine → hanya 1 berhasil, 19 error 23505

Database constraints eliminate the check-then-act vulnerability by merging check and write atomically.

Lima Mekanisme Constraint PostgreSQL

Database constraint menerapkan integritas data pada level storage

NOT NULL (23502)

Kolom wajib: email, username, user_id tidak boleh NULL
Constraint validation terjadi pada saat INSERT/UPDATE
Prevent integrity issues sebelum masuk ke business logic

CHECK (23514)

Predikat boolean: age >= 18, status IN ('active','inactive','banned')
total_cents > 0 mencegah order dengan nilai nol atau negatif
Validasi domain-specific berbasis aturan deklaratif

Satu kemunculan per nilai di kolom atau kombinasi kolom

Mencegah race condition read-then-write secara otomatis

B-tree index otomatis dibuat untuk performa lookup

Constraints provide declarative, ACID-guaranteed enforcement at the storage layer.

Referential Integrity & Partial Uniqueness

FOREIGN KEY dan partial unique index menambah fleksibilitas desain

FOREIGN KEY (23503)

Menjamin user_id di orders merujuk ke record users yang ada
Mencegah orphan records tanpa data induk yang valid
Constraint name tersedia di error detail untuk debugging

PARTIAL UNIQUE INDEX

CREATE UNIQUE INDEX ... WHERE deleted_at IS NULL
Uniqueness hanya untuk data aktif (deleted_at IS NULL)
Email dapat dipakai ulang setelah soft-delete

Active user dengan email same@example.com: hanya 1

Soft-deleted user dengan email same@example.com: diperbolehkan

Active user baru dengan email same@example.com: diperbolehkan

Partial unique index enables conditional uniqueness for soft-delete patterns.

SQLSTATE: Kode Error Standar PostgreSQL

Class 23: Constraint Violation — stable dan tidak terlokalisasi

KODE SQLSTATE

23502: NOT NULL violation — kolom wajib kosong
23503: FOREIGN KEY violation — referensi tidak ada
23505: UNIQUE violation — duplicate key
23514: CHECK violation — nilai di luar batas
23P01: Exclusion violation — exclusion constraint

ERROR OBJECT

ConstraintError{Code, ConstraintName, TableName, Detail}
ConstraintName: nama constraint yang dilanggar
TableName: tabel yang terpengaruh
Detail: pesan error detail untuk debugging

23505 → HTTP 409 Conflict (resource exists)

23502 → HTTP 400 Bad Request (missing field)

23514 → HTTP 422 Unprocessable Entity (validation failed)

23503 → HTTP 400 Bad Request (invalid reference)

SQLSTATE codes (23×xx) are stable across locales — use codes, not error text.

Proof: UnsafeStore vs SafeStore di Bawah Konkurensi

Perbandingan perilaku konkuren 20 goroutine dengan email sama

UNSAFE STORE (TANPA CONSTRAINT)

20 goroutine memanggil RegisterUser() dengan email 'unsafe@example.com'
Setiap goroutine melakukan SELECT → scan loop → Sleep(1ms) → INSERT
Hasil: GetUsersCount() > 1 (dua atau lebih record berhasil inserted)
Dua goroutine dapat membaca count=0 secara bersamaan

SAFE STORE (DENGAN CONSTRAINT)

20 goroutine memanggil RegisterUser() dengan email 'safe@example.com'
Database constraint 23505 menolak 19 goroutine secara atomik
Hasil: GetUsersCount() = 1 (hanya 1 goroutine berhasil)
Exactly 1 success + 19 SQLSTATE 23505 errors

Without constraint: race condition terjadi

With UNIQUE constraint: exactly one winner, N-1 failures

Database constraint mengeliminasi check-then-act window

Without atomic constraint enforcement, read-then-write patterns fail under concurrency.

Engine Simulator: In-Memory PostgreSQL Semantics

Simulator Go merepresentasikan semantik PostgreSQL tanpa Docker

KOMPONEN ENGINE

Engine: central state store dengan sync.RWMutex
users: map[int64]User, orders: map[int64]Order
InsertUser(): NOT NULL → CHECK → UNIQUE → Generate ID
InsertOrder(): NOT NULL → CHECK → FOREIGN KEY → Generate ID

COARSE-GRAINED LOCKING

e.mu.Lock() menahan untuk seluruh operasi insert
Critical section atomik: cek dan tulis dalam satu lock
Tidak ada interleaving antara check dan write

Coarse-grained lock bukan B-tree page latch

Readability dan zero-dependency diprioritaskan

Semantik SQLSTATE dan concurrency pattern termodel

Engine models SQLSTATE behavior and concurrency patterns — not production throughput.

Constraint Check Flow: Insert User

Urutan evaluasi constraint di Engine.InsertUser

URUTAN EVALUASI

1. NOT NULL: email, username, age, status wajib terisi
2. CHECK: age >= 18, status IN ('active','inactive','banned')
3. UNIQUE: email tidak boleh duplicate
4. Generate Primary Key dan commit ke users map
5. Update B-tree index simulasi

ATOMIC CRITICAL SECTION

e.mu.Lock() di awal operasi
Semua check dan write terjadi di dalam lock
e.mu.Unlock() setelah commit berhasil

InsertUserPartial() menerima flag usePartialIndex

Partial unique index: WHERE deleted_at IS NULL

Soft-deleted user tidak ter-index untuk uniqueness check

Constraints are evaluated atomically during INSERT — no window for interleaving.

Contoh Pelanggaran Constraint

Kode test menunjukkan pelanggaran masing-masing constraint

NOT NULL VIOLATION (23502)

RegisterUser{Username: 'alice', Age: 25} tanpa email → error
RegisterUser{Email: 'alice@example.com', Age: 25} tanpa username → error
CreateOrder{TotalCents: 100} tanpa user_id → error

CHECK VIOLATION (23514)

User age=16 → age < 18 violates CHECK constraint
User status='banned' → status tidak di ('active','inactive','banned')
Order total_cents=0 → total_cents <= 0 violates CHECK

First insert email='unique@example.com' → sukses

Second insert email='unique@example.com' → 23505 error

Concurrent insert same email → exactly 1 success, 19 failures

Each constraint type has a specific SQLSTATE code — use codes, not text.

Foreign Key Constraint & Referential Integrity

Orders harus memiliki user yang valid di database

INSERT ORDER FLOW

1. NOT NULL: user_id wajib terisi
2. CHECK: total_cents > 0
3. FOREIGN KEY: user_id harus ada di users map
4. Generate Primary Key dan commit ke orders map

VIOLASI FK

CreateOrder{UserID: 99999, TotalCents: 5000} → error 23503
99999 tidak ada di users map → referential integrity violation
Order terpisah dari user induknya (orphan record)

Insert User: id=1, email='fk@example.com'

CreateOrder{UserID: 1, TotalCents: 5000} → sukses

Order.id dan Order.UserID konsisten

Foreign key prevents orphan records — orders must reference existing users.

Partial Unique Index: Soft-Delete Uniqueness Pattern

Uniqueness hanya untuk record aktif (deleted_at IS NULL)

DDL INDEX

CREATE UNIQUE INDEX users_email_partial_idx ON users(email) WHERE deleted_at IS NULL
Index hanya memuat row dengan deleted_at IS NULL
Row dengan deleted_at NOT NULL tidak ter-index

SCENARIO: EMAIL REUSE

1. Insert active user: email='partial@example.com' → sukses
2. Insert duplicate active → 23505 (partially indexed)
3. Soft delete: user1.deleted_at = now()
4. Insert new active dengan email yang sama → sukses (user1 tidak ter-index)

Unique constraint: active users only

Partial index mencegah dupe aktif tanpa blokir soft-delete

User dapat di-activate ulang dengan email yang sama

Partial unique index: uniqueness only for active records, soft-deleted excluded.

Error Handling Pattern

Map SQLSTATE ke domain error dengan ConstraintError

CONSTRAINTERROR STRUCTURE

```
type ConstraintError struct {
Code SQLState
ConstraintName string
TableName string
Detail string
}
```

CONSTRUCTOR FUNCTIONS

```
NewNotNullViolation(table, column, constraintName)
NewUniqueViolation(table, constraintName, detail)
NewCheckViolation(table, constraintName, detail)
NewForeignKeyViolation(table, constraintName, detail)
```

23505 → 'conflict: resource with this unique attribute already exists'

23502 → 'invalid input: mandatory field is missing'

23514 → 'validation failed: value outside permissible boundary'

23503 → 'reference error: referenced entity does not exist'

Use SQLSTATE codes for error classification, not error text parsing.

Store Layer: UnsafeStore vs SafeStore

Pemisahan antara validasi aplikasi dan constraint database

UNSAFESTORE

RegisterUser: app-level check loop → Sleep → InsertUserUnsafe
InsertUserUnsafe bypass database constraint
Vulnerable: two goroutines dapat lolos dengan email yang sama

SAFESTORE

RegisterUser: delegate ke Engine.InsertUser
Database constraint menangani race condition atomik
All-or-nothing: success or 23505 error

NewUnsafeStore(eng)

NewSafeStore(eng)

Same engine, different constraint enforcement strategy

UnsafeStore demonstrates the race condition, SafeStore shows constraint protection.

Test Coverage: 8 Test Cases

Semua constraint type dan concurrency pattern teruji

BASIC CONSTRAINT TESTS

TestNotNullConstraints: 3 assertions untuk email/username/user_id
TestCheckConstraints: 3 assertions untuk age/status/total_cents
TestUniqueConstraint: duplicate email rejection
TestForeignKeyConstraint: orphan order rejection

ADVANCED TESTS

TestPartialUniqueIndex: 6 assertions untuk soft-delete pattern
TestConcurrentRegistration_Safe_EnforcesUniqueness: 20 goroutines, 1 success
TestConcurrentRegistration_Unsafe_SuffersRaceCondition: 20 goroutines, >1 record
TestErrorClassification: SQLSTATE mapping verification

go test -race ./... → clean, no data races

TestConcurrentRegistration_Safe_*: thread-safe constraint enforcement

TestConcurrentRegistration_Unsafe_*: data race terdeteksi

8 test cases cover all constraint types and concurrency edge cases.

Test Results: 8/8 PASS

go test -race ./... menghasilkan clean run tanpa race condition

TEST EXECUTION

TestNotNullConstraints → PASS
TestCheckConstraints → PASS
TestUniqueConstraint → PASS
TestForeignKeyConstraint → PASS
TestPartialUniqueIndex → PASS

CONCURRENCY TESTS

TestConcurrentRegistration_Safe_EnforcesUniqueness → PASS
TestConcurrentRegistration_Unsafe_SuffersRaceCondition → PASS
TestErrorClassification → PASS

Total test: 8 test cases

Pass rate: 100% (8/8 PASS)

Race detector: clean, no data races

Test time: ~1.5s

8/8 test cases PASS with race detector clean — constraint enforcement verified.

Demo Program: Interactive Test Harness

cmd/demo/main.go menggambarkan penggunaan Store layer

FEATURES

Register users dengan email dan password
Create orders dengan user reference
Stress test dengan 50 goroutines konkuren
Real-time output: successful/rejected counts

SAMPLE OUTPUT

Total Goroutine: 50
Successful: 1
Rejected 23505: 49
Integrity intact: true

go run ./cmd/demo

Interactive prompt untuk user input

Stress test mode untuk konkurensi test

Demo program illustrates real-world usage of constraint-protected store.

Production Migration Strategy

NOT VALID + VALIDATE CONSTRAINT pattern untuk zero-downtime

NOT VALID PATTERN

ALTER TABLE orders ADD CONSTRAINT fk_user FOREIGN KEY(user_id)
REFERENCES users(id) NOT VALID
Constraint tidak divalidasi untuk existing rows
Existing rows tidak dicek — constraint hanya berlaku untuk new/updated rows

VALIDATE CONSTRAINT

ALTER TABLE orders VALIDATE CONSTRAINT fk_user
Background check: scan existing rows tanpa table lock
Zero-downtime: application tetap can read/write

1. Add constraint NOT VALID

2. Validate constraint (background)

3. Drop NOT VALID if desired

4. Add indexes on referencing columns untuk DELETE/UPDATE

NOT VALID + VALIDATE avoids table lock and enables incremental validation.

Keterbatasan Database Constraints

Constraints hanya untuk row-scoped logic

ROW-SCOPED ONLY

CHECK cannot reference other rows
CHECK cannot query external services
CHECK cannot perform cross-row validation

CROSS-ROW INVARIANTS

Total balance = sum of accounts — requires aggregate
At most one active subscription per customer — requires count
Solution: SERIALIZABLE isolation atau triggers

Referencing columns tidak auto-indexed

DELETE/UPDATE pada referenced table mungkin full scan

Manually add index pada referencing columns

Constraints are row-scoped — cross-row invariants require different approaches.

Prinsip Desain: Defense in Depth

Kombinasi validasi aplikasi dan constraint database

TWO-LAYER DEFENSE

Layer 1 (Aplikasi): user-friendly error messages
Layer 2 (Database): absolute correctness guarantee
Constraint database = last line of defense

COMPLEMENTARY ROLES

Application validation: UX lapis pertama
Database constraint: safety net untuk integrity
Duas layer bekerja bersama, bukan menggantikan

Flight check-in: app checks (bag weight), security check (physical scan)

Both layers catch different types of issues

Security check = database constraint (last line)

Application for UX, database for correctness — defense in depth.

Kapan Gunakan Constraint Database vs App Validation

Konteks dan trade-off masing-masing pendekatan

USE DATABASE CONSTRAINT

Data integrity critical: UNIQUE, NOT NULL, CHECK
Race condition possible: concurrency workload
Single source of truth: database layer
Cross-application consistency: multi-service

USE APPLICATION VALIDATION

User-friendly error messages
Form validation on UI before submit
Performance: avoid DB round-trip untuk simple check
Complex business logic: multi-step validation

Always use database constraint untuk integrity

Use application validation untuk UX

Never rely on app validation alone

Database constraint = mandatory, app = optional

Database constraint for mandatory integrity, app validation for optional UX.

SQLSTATE Standards: PostgreSQL & SQLite

Class 23: Constraint Violation — stable dan portabel

SQLSTATE OVERVIEW

PostgreSQL: class 23Constraint Violation
SQLite: class 23Constraint Violation
SQL standard: ISO/IEC 9075-2
Code 23×xx tidak terlokalisasi

COMMON CODES

23502: NOT NULL violation
23503: FOREIGN KEY violation
23505: UNIQUE violation
23514: CHECK violation
23P01: Exclusion violation

Code stable across locales

Error text may be localized

Always use codes, never parse text

SQLSTATE codes are stable — use codes, not error text, for classification.

Batasan Implementasi Lab 27

Simulator Go vs PostgreSQL live

BATAS 1: IN-MEMORY ENGINE

Engine menggunakan Go map dan sync.RWMutex
Bukan PostgreSQL live — tidak ada B-tree page latch
Query counter tidak ada (tidak ada query execution)

BATAS 2: LOCKING STRATEGY

Coarse-grained table lock, bukan row-level lock
Bukan B-tree page latch atau MVCC
Trade-off: readability dan zero-dependency

Lab untuk pembelajaran konsep

Bukan benchmark production throughput

Verifikasi constraint behavior, bukan PostgreSQL internals

This lab proves constraint behavior conceptually — not production PostgreSQL.

Prinsip Desain Inti

Tiga prinsip untuk data integrity yang benar

PRINSIP 1: DELEGATE TO DATABASE

Delegasikan integrity check ke database constraint
Database constraint = single source of truth
Avoid check-then-act pattern di application layer

PRINSIP 2: DEFENSE IN DEPTH

Application validation untuk UX (optional)
Database constraint untuk correctness (mandatory)
Jangan gantikan constraint dengan app validation

Test konkuren eksplisit dengan go test -race

Stress test dengan goroutines untuk race detection

Sequential test tidak dapat mendeteksi race condition

Three principles: delegate to DB, defense in depth, concurrent test.

Ringkasan Lab 27

Database constraints provide atomic integrity enforcement — no window for race conditions.

KEY RESULTS

5 constraint types: NOT NULL, CHECK, UNIQUE, FOREIGN KEY, PARTIAL UNIQUE

SQLSTATE codes: 23502/23503/23505/23514/23P01

Race condition prevention via atomic constraint enforcement

REPOSITORY

https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/27-database-constraints

Lukman / lukman-ss

MENTAL MODEL

Database constraint eliminates check-then-act vulnerability

Two-layer defense: app for UX, DB for correctness

Partial unique index enables soft-delete email reuse

RESOURCES

Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/27-database-constraints

Commit SHA: 03a693cc41fcccbc02e6a1d03c0f0cf2c49dacff

Run tests: go test -v -race ./...

Run demo: go run ./cmd/demo